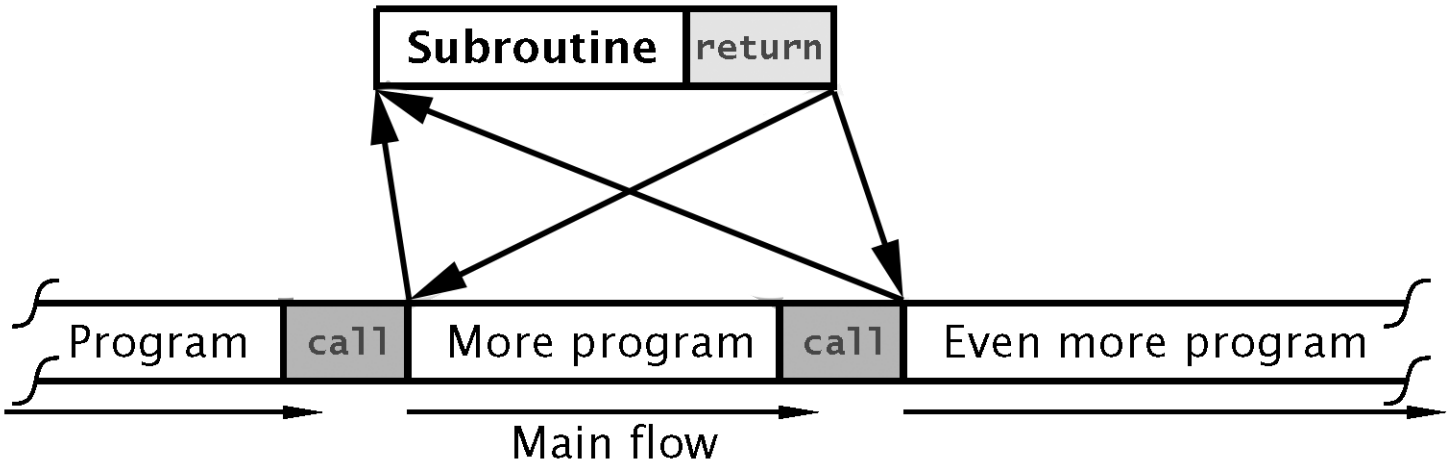
3.6 Subroutine call and return

Subroutine

- A subroutine is a sequence of instructions that can be called from different places in a program.
- Two reasons for creating subroutines:
  - The problem is too big: Easier to divide the problem into smaller sub-problems
  - There are several places in a program that need to perform the same operation

Calling subroutine: Is it just branching?



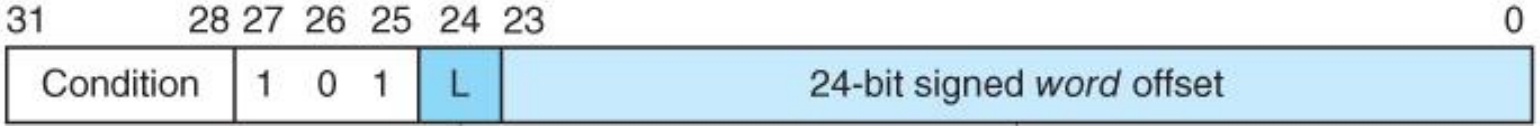
- Calling subroutine involves the “jumping” part, which is the same as branching.
- But we need to get back to the main program after the subroutine returns.
- Thus, we need a location in which the return address can be stored.

Image courtesy of S. Katzen, The essential PIC18 Microcontroller, Springer

ARM support for subroutine

- Use `BL` (branch with link)
  - Performing branching and
  - Store the return address (i.e., the address of the instruction immediately below the bl instruction) into `r14` (also called `lr`) register.

FIGURE 3.41 Encoding ARM's branch and branch-with-link instructions



- At the end of the subroutine, use `BX lr` to return [lr] to [pc].

Example 1

Suppose that you want to evaluate
if x > 0 then x = 16x + 1 else x = 32x
several times in a program. Assuming that x is in r0, we can write:

```
Func1:
cmp r0, #0 //test for x > 0
movgt r0, r0, lsl #4 // if x > 0 x = 16x
addgt r0, r0, #1 // if x > 0 then x = 16x + 1
movle r0, r0, lsl #5 // ELSE if x <= 0 THEN x = 32x
bx lr // [pc] <- [lr]
```

Example 1

```
6  .text
7  .global _start
8  _start:
9
10 ldr r0, =-1
11
12 bl Func1
13
14 ldr r4, =0x1000
15 str r0, [r4] // store the result in address 1000
16
17 Here:
18 b Here
19
20 Func1:
21 cmp r0, #0 //test for x > 0
22 movgt r0, r0, lsl #4 // if x > 0 x = 16x
23 addgt r0, r0, #1 // if x > 0 then x = 16x + 1
24 movle r0, r0, lsl #5 // ELSE if x < 0 THEN x = 32x
25 bx lr // [pc] <- [lr]
26
27 .end
```

Instructions calling the subroutine Func1:

- Set up r0 as the input parameter
- After Func1 generates an output stored in r0, copy the output to address 1000

Note that the subroutine appears after the `Here: b Here` instruction. This guarantees that the subroutine will not be executed by mistake.

Example 1

Address	Machine Code	
		10 ldr r0, =-1
		_start:
00000000	e3e00000	mvn r0, #0 ; 0x0
		12 bl Func1
00000004	eb000002	bl 0x14 (0x14: Func1)
		14 ldr r4, =0x1000
00000008	e3a04a01	mov r4, #4096 ; 0x1000
		15 str r0, [r4] // store the result in address 1000
0000000c	e5840000	str r0, [r4]
		17 Here:
		18 b Here
		Here:
00000010	eafffffe	b 0x10 (0x10: Here)
		20 Func1:
		21 cmp r0, #0 //test for x > 0
		Func1:
00000014	e3500000	cmp r0, #0 ; 0x0
		22 movgt r0, r0, lsl #4 // if x > 0 x = 16x
00000018	c1a00200	lslgt r0, r0, #4
		23 addgt r0, r0, #1 // if x > 0 then x = 16x + 1
0000001c	c2800001	addgt r0, r0, #1 ; 0x1
		24 movle r0, r0, lsl #5 // ELSE if x < 0 THEN x = 32x
00000020	d1a00280	lslle r0, r0, #5
		25 bx lr // [pc] <- [lr]
00000024	e12fff1e	bx lr
		_end:

- After execution of the `bl` instruction:
  - [lr] = 0x08
  - [pc] = 0x14
- After execution of the bx instruction inside the subroutine Func1:
  - [pc] = 0x08

  thus returning to the line immediately below the `bl` instruction.
- Encoding at the `bl` instruction

$$\frac{Dest\ -Current\ PC}{4} = \frac{0x14\ -0x0C}{4} = 2$$

- Encoded Address = 000002

Example 2 (Lab 5)

1. Instructions calling the swap subroutine

```
14 .text        // Code section
15 .global _start
16 _start:
17
18 ldr    r0, =x   // store location of x
19 ldr    r1, =y   // store location of y
20 bl     swap     // call the swap subroutine
21
22 _stop:
23 b      _stop
```

2. swap subroutine

```
26 swap:
27 /*
28 -----------------------------------------------------
29 Swaps location of two values in memory.
30 Equivalent of: swap(*x, *y)
31 -----------------------------------------------------
32 Parameters:
33   [r0] - address of x
34   [r1] - address of y
35 Uses:
36   [r2] - value of x
37   [r3] - value of y
38 -----------------------------------------------------
39 */
40 stmfd   sp!, {r0-r3}    // preserve all registers
41
42 ldr     r2, [r0]        // get value at x
43 ldr     r3, [r1]        // get value at y
44 str     r2, [r1]        // store value of x in y
45 str     r3, [r0]        // store value of y in x
46
47 ldmfd   sp!, {r0-r3}    // pop preserved registers
48 bx      lr              // return from subroutine
```

3. Data storage

```
51 .data
52 .align
53 x:
54 .word    9
55 y:
56 .word    4
57
58 .end
```

Example 2 (Lab 5)

```
                                20  bl      swap      // call the s
00001008    eb000000                bl   0x1010   (0x1010: swap)
                                22  _stop:
                                23  b       _stop
                                _stop:
0000100c    eafffffe                b    0x100c   (0x100c: _stop)
                                25  //-------------------------
                                26  swap:
                                27  /*
                                28  -----------------------------
                                29  Swaps location of two values
                                30  Equivalent of: swap(*x, *y)
                                31  -----------------------------
                                32  Parameters:
                                33  [r0] - address of x
                                34  [r1] - address of y
                                35  Uses:
                                36  [r2] - value of x
                                37  [r3] - value of y
                                38  -----------------------------
                                39  */
                                40  stmfd   sp!, {r0-r3}    // pr
                                swap:
00001010    e92d000f                push    {r0, r1, r2, r3}
```

- After execution of the `bl` instruction:
  - [lr] = 0x100C
  - [pc] = 0x1010
- After execution of the bx instruction inside the subroutine swap:
  - [pc] = 0x100C

Preserving Registers

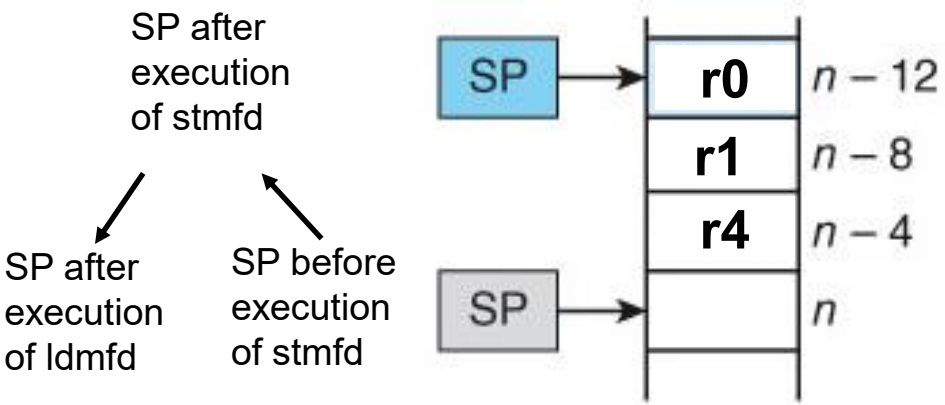
- The values in registers should be the same before and after a subroutine call, except registers being updated (e.g., r0 in Example 1) or those used to return a value.
- Registers are preserved and recovered by pushing and popping their values to the stack.
- ARM provides instructions for pushing and popping multiple register values at once by stmfd and ldmfd.

Block data transfer

stmfd (Store Memory Full Descending stack)
- copies a block of data from registers to memory

ldmfd (Load Memory Full Descending stack)
- copies a block of data from memory to registers

```
stmfd sp!, {r0,r1,r4}
ldmfd sp!, {r0,r1,r4}
```



The FD option:
- Full
  - SP points to the top item of stack
- Descending
  - stack grows towards lower addresses

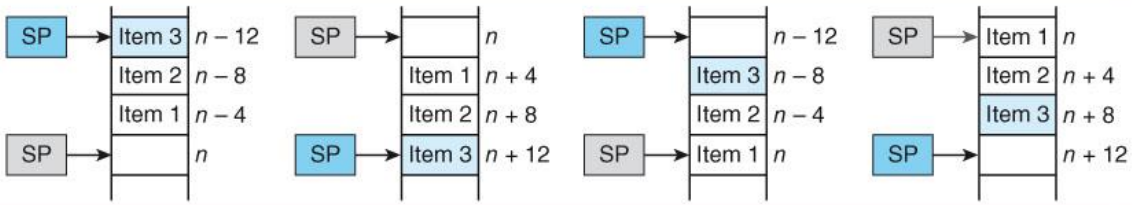
Note that sp moves only if appended with an !

Block data transfer

The full descending (FD) option is most often used in this course, but you should know about other options:

- Full vs. Empty
  - Full: SP points to the top item of the stack
  - Empty: SP points to the next empty element above the top of stack
- Descending vs. Ascending
  - The stack grows towards either lower (descending) or higher (ascending) addresses.

TABLE 3.5 Stack Types and the ARM Block Move Instruction Suffixes



Stack growth	Descending	Ascending	Descending	Ascending
Class	Full	Full	Empty	Empty
Stack suffix	FD	FA	ED	EA
Load suffix	IA (increment after)	DA (decrement after)	IB (increment before)	DB (decrement before)
Store suffix	DB (decrement before)	IB (increment before)	DA (decrement after)	IA (increment after)

© Cengage Learning 2014

Block data transfer

stmfd = stmdb

db = decrement before

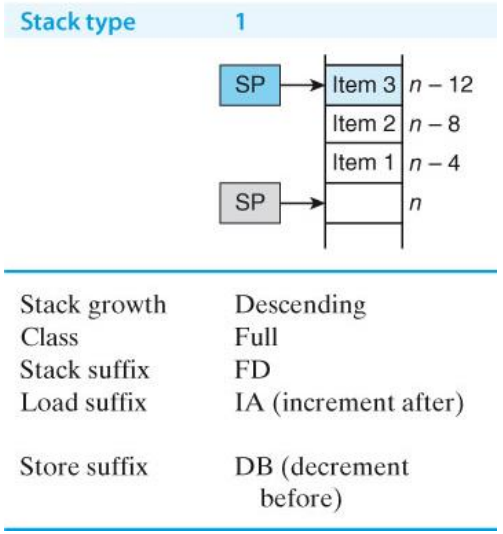
- SP decrements before storing the next element

ldmfd = ldmia

ia = increment after

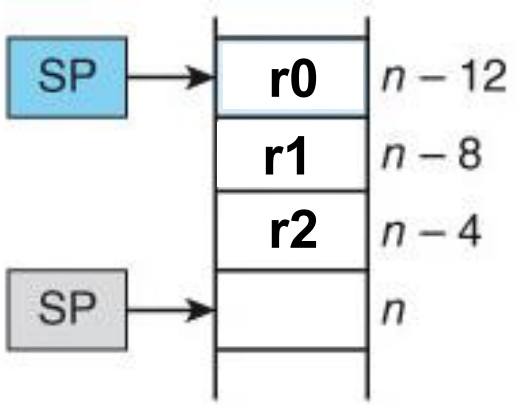
- SP increments after popping each element

Try to understand equivalence in other types of stacks.



Block data transfer

```
stmfd sp!, {r0-r2}
```



- If multiple registers are adjacent, they can be listed with a dash (-):
  - e.g., `stmfd sp!, {r0-r2}` pushes r0, r1 and r2.
- Note that the registers are pushed in right-to-left order (i.e., pushing r2, then r1 then r0)
- The same would happen even with `stmfd sp!, {r2, r0, r1}`
- If you require the stack to be in a different order, you must issue separate `stmfd` instructions.

🧠 Micro-Challenge: Block data transfer

```
mov sp, #0
mov r1, #0xEE
mov r2, #0x77
mov r3, #0xDD
mov r4, #0x44

stmfd sp!, {r2, r4, r3, r1}
ldmfd sp!, {r1-r2}
```

Express your answer in **32-bit hexadecimal format. No spaces between the hexadecimal digits. No '0x' prefix.** Again, no marks for incorrect format.

(a) What are the values of r1 and r2 after the execution of the code fragment?

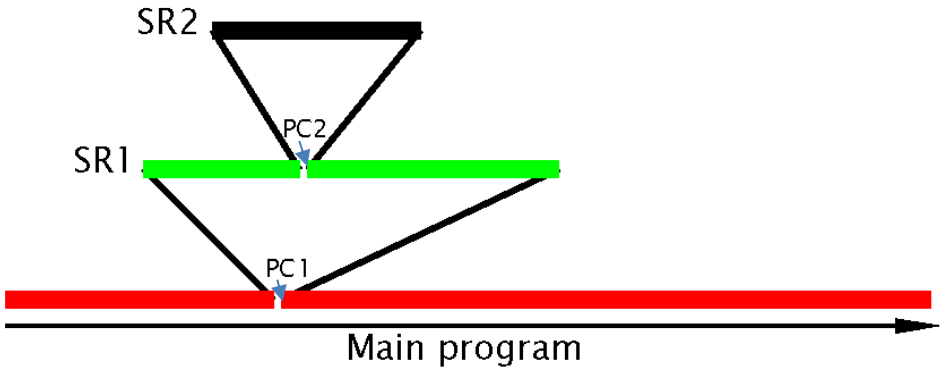
(b) What is [sp] (i.e., the address that is pointed to by sp) after the execution of the stmfd instruction?

(c) What is [sp] (i.e., the address that is pointed to by sp) after the execution of the ldmfd instruction?

Preserving registers in the swap subroutine

```
26 swap:
27 /*
28 ------------------------------------------------
29 Swaps location of two values in memory.
30 Equivalent of: swap(*x, *y)
31 ------------------------------------------------
32 Parameters:
33   [r0] - address of x
34   [r1] - address of y
35 Uses:
36   [r2] - value of x
37   [r3] - value of y
38 ------------------------------------------------
39 */
40 stmfd   sp!, {r0-r3}     // preserve all registers
41
42 ldr     r2, [r0]         // get value at x
43 ldr     r3, [r1]         // get value at y
44 str     r2, [r1]         // store value of x in y
45 str     r3, [r0]         // store value of y in x
46
47 ldmfd   sp!, {r0-r3}     // pop preserved registers
48 bx      lr               // return from subroutine
```

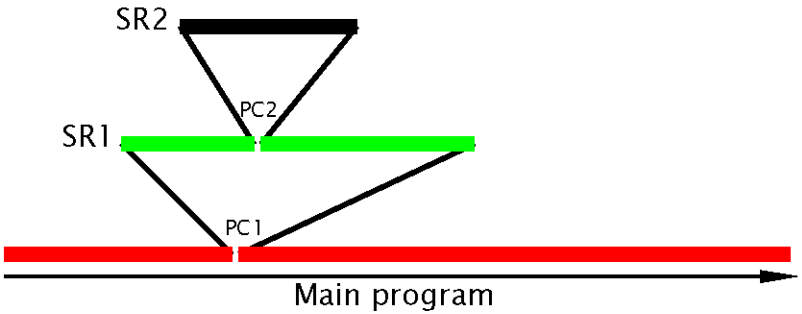
Nested subroutine



- Nested subroutine: calling a subroutine within a subroutine.
- BL call from Main saves the return address (PC1) to the link register (lr).
- BL call from SR1 saves the return address (PC2) to the lr, thereby overwriting PC1 → SR1 cannot return to Main.

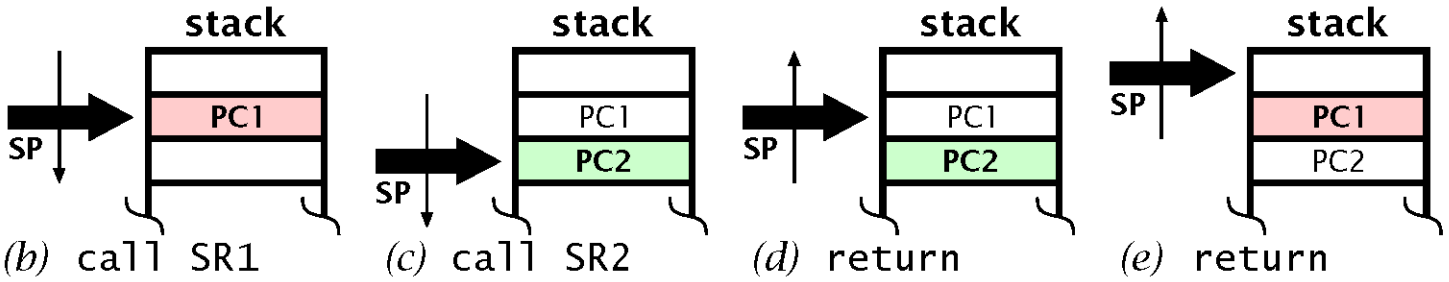
Nested subroutine

Some microcontrollers have a dedicated stack to handle nested subroutines, but not ARM.



(a) Two-deep nesting

SP: Stack Pointer
TOS: Top of Stack
(The address pointed to by SP)



TOS = PC1
SP = 1

TOS = PC2
SP = 2

PC = PC2
TOS = PC1
SP = 1

PC = PC1
SP = 0

How does ARM handle nested subroutine calls?

- In ARM, no such stack. Need to create our own stack.
- A leaf routine doesn't call another routine (at the end of tree).
  - If you call a leaf routine with BL, the return address is saved in link register r14. A return to the calling point is made with `bx lr`
- If the routine is not a leaf routine, you cannot call another routine without first saving the link register.

```
        BL      XYZ                 ;call a simple leaf routine
        .
        BL      XYZ1                ;call a routine that calls a nested routine
        .
XYZ     . . .                       ;code (this is the leaf routine)
        .
        bx lr                       ;copy link register into PC and return
XYZ1    STMFD   sp!,{r0-r4,lr}      ;save working registers and link register
        .
        BL      XYZ                 ;call XZY – overwrites the old link register
        .
        LDMFD   sp!,{r0-r4,pc}      ;restore registers and force a return
```

- The link register (lr) is preserved just as other registers
- At the end of the subroutine the preserved lr is popped to pc.

Example of a nested subroutine

- The following program calculates the value of f(0x07) where
  - f(x) = g(x+0x18) + 0x05
  - g(x) = x+0x12
- Determine the status of the stack at Points 1-4.

Example of a nested subroutine

```
6  .text
7  .global _start
8  _start:
9
10 ldr sp, =0
11 ldr r1, =0x07
12 bl f
13 ldr r2, =Result
14 str r0, [r2]
15
16 Here:
17 b Here
18
19
20 f: // r1 = input parameter, r0 = output value
21 stmfd sp!, {r1, lr} // Point 1
22 add r1, r1, #0x18
23 bl g
24 add r0, r0, #0x05
25 ldmfd sp!, {r1, pc} // Point 4
26
27 g: // r1 = input parameter, r0 = output value
28 stmfd sp!, {r1, lr} // Point 2
29 add r0, r1, #0x12
30 ldmfd sp!, {r1, pc} // Point 3
31
32 .data
33 .bss
34 Result:
35 .space 4
36
37 .end
```

Could do this because g is a leaf subroutine, but saving to stack is always safe:

```
g:
stmfd sp!, {r1}
add r0, r1, #0x12
ldmfd sp!, {r1}
bx lr
```

Example of a nested subroutine

```
Address   Opcode     Disassembly
                     _start:
00000000  e3a0d000       mov     sp, #0  ; 0x0
                     11  ldr r1, =0x07
00000004  e3a01007       mov     r1, #7  ; 0x7
                     12  bl f
00000008  eb000002       bl  0x18  (0x18: f)
                     13  ldr r2, =Result
0000000c  e59f2024       ldr     r2, [pc, #36]   ; 0x38
00000010  e5820000   14  str     r0, [r2]
                     16  Here:
                     17  b Here
                     Here:
00000014  eafffffe       b   0x14  (0x14: Here)
                     20  f: // r1 = input parameter, r0 =
                     21  stmfd sp!, {r1, lr} // Point 1
                     f:
00000018  e92d4002       push    {r1, lr}
                     22  add r1, r1, #0x18
0000001c  e2811018       add     r1, r1, #24 ; 0x18
                     23  bl g
00000020  eb000001       bl  0x2c  (0x2c: g)
                     24  add r0, r0, #0x05
00000024  e2800005       add     r0, r0, #5  ; 0x5
                     25  ldmfd sp!, {r1, pc} // Point 4
00000028  e8bd8002       pop     {r1, pc}
                     27  g: // r1 = input parameter, r0 =
                     28  stmfd sp!, {r1, lr} // Point 2
                     g:
0000002c  e92d4002       push    {r1, lr}
                     29  add r0, r1, #0x12
00000030  e2810012       add     r0, r1, #18 ; 0x12
                     30  ldmfd sp!, {r1, pc} // Point 3
00000034  e8bd8002       pop     {r1, pc}
```

Point 1

SP →

Address	Content
FFFFFFF8	00000007
FFFFFFFC	0000000C

Point 2

SP →

Address	Content
FFFFFFF0	0000001F
FFFFFFF4	00000024
FFFFFFF8	00000007
FFFFFFFC	0000000C

Point 3

SP →

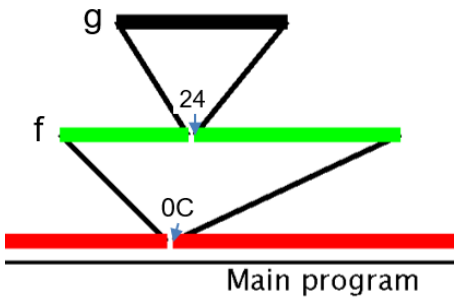
Address	Content
FFFFFFF8	00000007
FFFFFFFC	0000000C

[PC] = 00000024
[r1] = 0000001F

Point 4

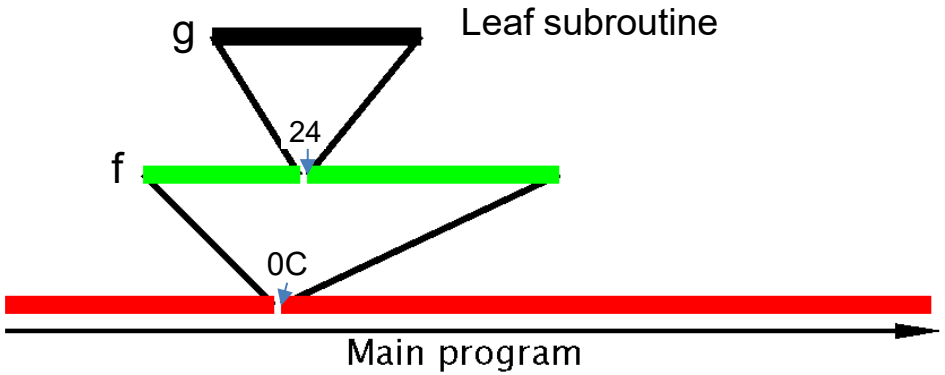
[PC] = 0000000C
[r1] = 00000007

Structure of the nested call



Note that [r1] is preserved
(i.e., the bl call does not change its value)

Further notes



- `bx lr` can be used only in leaf subroutine (e.g., `g`)
  - Saving `lr` in stack is safe for all subroutines though.
- What happens if `f` is returned by `bx lr`?
  - Upon completion of f, program should return to address 0C
  - After `f` is called, [lr] = 0C
  - After `g` is called, [lr] = 24, overwriting 0C
  - Thus, `f` cannot be returned by `bx lr`.